

```

        auto hostName = ref new
Windows::Networking::HostName(L"localhost");

        this->clientListBox->Items->Append(L"client is trying to
connect...");

        Concurrency::create_task(this->streamSocket-
>ConnectAsync(hostName, L"1337")).then(
            [=](Concurrency::task< void >)
            {
                this->clientListBox->Items->Append(L"client connected");

                // Send a request to the echo server.
                auto dataWriter = ref new DataWriter(this->streamSocket-
>OutputStream);
                auto request = ref new Platform::String(L"Hello, World!");
                dataWriter->WriteUInt32(request->Length());
                dataWriter->WriteString(request);

                Concurrency::create_task(dataWriter->StoreAsync()).then(
                    [=](Concurrency::task< unsigned int >)
                    {
                        std::wstringstream wstringstream;
                        wstringstream << L"client sent the request: \"" <<
request->Data() << L"\"";
                        this->clientListBox->Items->Append(ref new
Platform::String(wstringstream.str().c_str()));

                        Concurrency::create_task(dataWriter->FlushAsync()).then(
                            [=](Concurrency::task< bool >)
                            {
                                dataWriter->DetachStream();

                                // Read data from the echo server.
                                auto dataReader = ref new DataReader(this-
>streamSocket->InputStream);
                                Concurrency::create_task(dataReader-
>LoadAsync(sizeof(unsigned int))).then(
                                    [=](unsigned int bytesLoaded)
                                    {
                                        unsigned int stringLength = dataReader-
>ReadUInt32();

                                        Concurrency::create_task(dataReader-
>LoadAsync(stringLength)).then(
                                            [=](unsigned int bytesLoaded)
                                            {
                                                Platform::String^ response = dataReader-
>ReadString(bytesLoaded);

                                                this->Dispatcher-
>RunAsync(CoreDispatcherPriority::Normal, ref new DispatchedHandler(
                                                    [=]
                                                    {
                                                        std::wstringstream wstringstream;
                                                        wstringstream << L"client received the
response: \"" << response->Data() << L"\"";

```

```

        this->clientListBox->Items->Append(ref
new Platform::String(wstringstream.str().c_str()));

        delete this->streamSocket;
        this->streamSocket = nullptr;

        this->clientListBox->Items-
>Append(L"client closed its socket");
    }));
    });
    });
    });
    });
    });
    });
    }
    catch (Platform::Exception^ ex)
    {
        Windows::Networking::Sockets::SocketErrorStatus webErrorStatus =
Windows::Networking::Sockets::SocketError::GetStatus(ex->HRESULT);
        this->serverListBox->Items->Append(webErrorStatus.ToString() !=
L"Unknown" ? webErrorStatus.ToString() : ex->Message);
    }
}

```

References to StreamSockets in C++ PPL continuations (applies to C++/CX, primarily)

ⓘ Note

If you use C++/WinRT coroutines, and you pass parameters by value, then this issue doesn't apply. For parameter-passing recommendations, see [Concurrency and asynchronous operations with C++/WinRT](#).

A **StreamSocket** remains alive as long as there's an active read/write on its input/output stream (let's take for example the **StreamSocketListenerConnectionReceivedEventArgs.Socket** that you have access to in your **StreamSocketListener.ConnectionReceived** event handler). When you call **DataReader.LoadAsync** (or **ReadAsync/WriteAsync/StoreAsync**), then that holds a reference to the socket (via the socket's input stream) until the **Completed** event handler (if any) of the **LoadAsync** is done executing.

The Parallel Patterns Library (PPL) doesn't schedule task continuations inline by default. In other words, adding a continuation task (with **task::then()**) doesn't guarantee that the continuation task will execute inline as the completion handler.

C++

```
void
StreamSocketListener_ConnectionReceived(Windows::Networking::Sockets::Stream
SocketListener^ sender,
Windows::Networking::Sockets::StreamSocketListenerConnectionReceivedEventArg
s^ args)
{
    auto dataReader = ref new DataReader(args->Socket->InputStream);
    Concurrency::create_task(dataReader->LoadAsync(sizeof(unsigned
int))).then(
        [=](unsigned int bytesLoaded)
        {
            // Work in here isn't guaranteed to execute inline as the completion
            handler of the LoadAsync.
        });
}
```

From the perspective of the **StreamSocket**, the completion handler is done executing (and the socket is eligible for disposal) before the continuation body runs. So, to keep your socket from being disposed if you want to use it inside that continuation, you need to either reference the socket directly (via lambda capture) and use it, or indirectly (by continuing to access `args->Socket` inside continuations), or force continuation tasks to be inline. You can see the first technique (lambda capture) in action in the [StreamSocket sample](#). The C++/CX code in the [Build a basic TCP socket client and server](#) section above uses the second technique—it echoes the request back as a response, and it accesses `args->Socket` from within one of the innermost continuations.

The third technique is appropriate when you're not echoing a response back. You use the `task_continuation_context::use_synchronous_execution()` option to force PPL to execute the continuation body inline. Here's a code example showing how to do it.

C++

```
void
StreamSocketListener_ConnectionReceived(Windows::Networking::Sockets::Stream
SocketListener^ sender,
Windows::Networking::Sockets::StreamSocketListenerConnectionReceivedEventArg
s^ args)
{
    auto dataReader = ref new DataReader(args->Socket->InputStream);

    Concurrency::create_task(dataReader->LoadAsync(sizeof(unsigned
int))).then(
        [=](unsigned int bytesLoaded)
        {
            unsigned int messageLength = dataReader->ReadUInt32();
            Concurrency::create_task(dataReader->LoadAsync(messageLength)).then(
                [=](unsigned int bytesLoaded)
            {
            });
        });
}
```

```

    {
        Platform::String^ request = dataReader->ReadString(bytesLoaded);
        this->Dispatcher->RunAsync(CoreDispatcherPriority::Normal, ref
new DispatchedHandler(
    [=]
    {
        std::wstringstream wstringstream;
        wstringstream << L"server received the request: \"" <<
request->Data() << L"\"";
        this->serverListBox->Items->Append(ref new
Platform::String(wstringstream.str().c_str()));
    }));
    }, Concurrency::task_continuation_context::use_synchronous_execution());
}

```

This behavior applies to all of the sockets and WebSockets classes in the [Windows.Networking.Sockets](#) namespace. But client-side scenarios usually store sockets in member variables, so the issue is most applicable to the [StreamSocketListener.ConnectionReceived](#) scenario, as illustrated above.

Build a basic UDP socket client and server

A UDP (User Datagram Protocol) socket is similar to a TCP socket in that it also provides low-level network data transfers in either direction. But, while a TCP socket is for long-lived connections, a UDP socket is for applications where an established connection is not required. Because UDP sockets don't maintain connection on both endpoints, they're a fast and simple solution for networking between remote machines. But UDP sockets don't ensure integrity of the network packets nor even whether packets make it to the remote destination at all. So your app will need to be designed to tolerate that. Some examples of applications that use UDP sockets are local network discovery, and local chat clients.

To demonstrate basic UDP operations, the example code below shows the [DatagramSocket](#) class being used to both send and receive data over UDP to form an echo client and server. Create a new project, and put both the client and the server code below into the same project. Just as for a TCP socket, you'll need to declare the **Private Networks (Client & Server)** app capability.

An echo client and server, using UDP sockets

Construct a [DatagramSocket](#) to play the role of the echo server, bind it to a specific port number, listen for an incoming UDP message, and echo it back. The

DatagramSocket.MessageReceived event is raised when a message is received on the socket.

Construct another **DatagramSocket** to play the role of the echo client, bind it to a specific port number, send a UDP message, and receive a response.

Create a new **Page** named `DatagramSocketPage`. Put the XAML markup in `DatagramSocketPage.xaml`, and then put the imperative code inside the `DatagramSocketPage` class.

XAML

```
<Grid Background="{ThemeResource ApplicationPageBackgroundThemeBrush}">
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="*/>
    </Grid.RowDefinitions>

    <StackPanel>
        <TextBlock Margin="9.6,0" Style="{StaticResource TitleTextBlockStyle}" Text="UDP socket example"/>
        <TextBlock Margin="7.2,0,0,0" Style="{StaticResource HeaderTextBlockStyle}" Text="DatagramSocket"/>
    </StackPanel>

    <Grid Grid.Row="1">
        <Grid.RowDefinitions>
            <RowDefinition/>
            <RowDefinition/>
        </Grid.RowDefinitions>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="*/>
            <ColumnDefinition Width="*/>
        </Grid.ColumnDefinitions>
        <TextBlock Margin="9.6" Style="{StaticResource SubtitleTextBlockStyle}" Text="client"/>
        <ListBox x:Name="clientListBox" Grid.Row="1" Margin="9.6"/>
        <TextBlock Grid.Column="1" Margin="9.6" Style="{StaticResource SubtitleTextBlockStyle}" Text="server"/>
        <ListBox x:Name="serverListBox" Grid.Column="1" Grid.Row="1" Margin="9.6"/>
    </Grid>
</Grid>
```

C#

```
// Every protocol typically has a standard port number. For example, HTTP is
// typically 80, FTP is 20 and 21, etc.
// For this example, we'll choose different arbitrary port numbers for
// client and server, since both will be running on the same machine.
static string ClientPortNumber = "1336";
```

```

static string ServerPortNumber = "1337";

protected override void OnNavigatedTo(NavigationEventArgs e)
{
    this.StartServer();
    this.StartClient();
}

private async void StartServer()
{
    try
    {
        var serverDatagramSocket = new
Windows.Networking.Sockets.DatagramSocket();

        // The ConnectionReceived event is raised when connections are
received.
        serverDatagramSocket.MessageReceived +=
ServerDatagramSocket_MessageReceived;

        this.serverListBox.Items.Add("server is about to bind...");

        // Start listening for incoming TCP connections on the specified
port. You can specify any port that's not currently in use.
        await
serverDatagramSocket.BindServiceNameAsync(DatagramSocketPage.ServerPortNumbe
r);

        this.serverListBox.Items.Add(string.Format("server is bound to port
number {0}", DatagramSocketPage.ServerPortNumber));
    }
    catch (Exception ex)
    {
        Windows.Networking.Sockets.SocketErrorStatus webErrorStatus =
Windows.Networking.Sockets.SocketError.GetStatus(ex.GetBaseException()).HResu
lt);
        this.serverListBox.Items.Add(webErrorStatus.ToString() != "Unknown"
? webErrorStatus.ToString() : ex.Message);
    }
}

private async void
ServerDatagramSocket_MessageReceived(Windows.Networking.Sockets.DatagramSock
et sender, Windows.Networking.Sockets.DatagramSocketMessageReceivedEventArgs
args)
{
    string request;
    using (DataReader dataReader = args.GetDataReader())
    {
        request =
dataReader.ReadString(dataReader.UnconsumedBufferLength).Trim();
    }

    await this.Dispatcher.RunAsync(CoreDispatcherPriority.Normal, () =>
this.serverListBox.Items.Add(string.Format("server received the request: \"

```

```

{0}\", request)));

    // Echo the request back as the response.
    using (Stream outputStream = (await
sender.GetOutputStreamAsync(args.RemoteAddress,
DatagramSocketPage.ClientPortNumber)).AsStreamForWrite())
    {
        using (var streamWriter = new StreamWriter(outputStream))
        {
            await streamWriter.WriteLineAsync(request);
            await streamWriter.FlushAsync();
        }
    }

    await this.Dispatcher.RunAsync(CoreDispatcherPriority.Normal, () =>
this.serverListBox.Items.Add(string.Format("server sent back the response:
\"{0}\", request)));

    sender.Dispose();

    await this.Dispatcher.RunAsync(CoreDispatcherPriority.Normal, () =>
this.serverListBox.Items.Add("server closed its socket"));
}

private async void StartClient()
{
    try
    {
        // Create the DatagramSocket and establish a connection to the echo
server.
        var clientDatagramSocket = new
Windows.Networking.Sockets.DatagramSocket();

        clientDatagramSocket.MessageReceived +=
ClientDatagramSocket_MessageReceived;

        // The server hostname that we will be establishing a connection to.
In this example, the server and client are in the same process.
        var hostName = new Windows.Networking.HostName("localhost");

        this.clientListBox.Items.Add("client is about to bind...");

        await
clientDatagramSocket.BindServiceNameAsync(DatagramSocketPage.ClientPortNumbe
r);

        this.clientListBox.Items.Add(string.Format("client is bound to port
number {0}", DatagramSocketPage.ClientPortNumber));

        // Send a request to the echo server.
        string request = "Hello, World!";
        using (var serverDatagramSocket = new
Windows.Networking.Sockets.DatagramSocket())
        {
            using (Stream outputStream = (await

```

```

serverDatagramSocket.GetOutputStreamAsync(hostName,
DatagramSocketPage.ServerPortNumber)).AsStreamForWrite())
    {
        using (var streamWriter = new StreamWriter(outputStream))
        {
            await streamWriter.WriteLineAsync(request);
            await streamWriter.FlushAsync();
        }
    }
}

this.clientListBox.Items.Add(string.Format("client sent the request:
\"{0}\"", request));
}
catch (Exception ex)
{
    Windows.Networking.Sockets.SocketErrorStatus webErrorStatus =
Windows.Networking.Sockets.SocketError.GetStatus(ex.GetBaseException().HResult);
    this.clientListBox.Items.Add(webErrorStatus.ToString() != "Unknown"
? webErrorStatus.ToString() : ex.Message);
}
}

private async void
ClientDatagramSocket_MessageReceived(Windows.Networking.Sockets.DatagramSocket sender, Windows.Networking.Sockets.DatagramSocketMessageReceivedEventArgs args)
{
    string response;
    using (DataReader dataReader = args.GetDataReader())
    {
        response =
dataReader.ReadString(dataReader.UnconsumedBufferLength).Trim();
    }

    await this.Dispatcher.RunAsync(CoreDispatcherPriority.Normal, () =>
this.clientListBox.Items.Add(string.Format("client received the response: \"{0}\"", response)));

    sender.Dispose();

    await this.Dispatcher.RunAsync(CoreDispatcherPriority.Normal, () =>
this.clientListBox.Items.Add("client closed its socket"));
}

```

C++/WinRT

```

#include <winrt/Windows.Foundation.h>
#include <winrt/Windows.Networking.Sockets.h>
#include <winrt/Windows.Storage.Streams.h>
#include <winrt/Windows.UI.Core.h>
#include <winrt/Windows.UI.Xaml.Navigation.h>

```

```

#include <sstream>

using namespace winrt;
using namespace Windows::Foundation;
using namespace Windows::Storage::Streams;
using namespace Windows::UI::Core;
using namespace Windows::UI::Xaml::Navigation;
...
private:
    Windows::Networking::Sockets::DatagramSocket m_clientDatagramSocket;
    Windows::Networking::Sockets::DatagramSocket m_serverDatagramSocket;

public:
    void OnNavigatedTo(NavigationEventArgs const& /* e */)
    {
        StartServer();
        StartClient();
    }

private:
    IAsyncAction StartServer()
    {
        try
        {
            // The ConnectionReceived event is raised when connections are
            received.
            m_serverDatagramSocket.MessageReceived({ this,
&DatagramSocketPage::ServerDatagramSocket_MessageReceived });

            serverListBox().Items().Append(winrt::box_value(L"server is
about to bind..."));

            // Start listening for incoming TCP connections on the specified
            port. You can specify any port that's not currently in use.
            co_await m_serverDatagramSocket.BindServiceNameAsync(L"1337");
            serverListBox().Items().Append(winrt::box_value(L"server is
bound to port number 1337"));
        }
        catch (winrt::hresult_error const& ex)
        {
            Windows::Networking::Sockets::SocketErrorStatus webErrorStatus{
Windows::Networking::Sockets::SocketError::GetStatus(ex.to_abi()) };
            serverListBox().Items().Append(webErrorStatus !=
Windows::Networking::Sockets::SocketErrorStatus::Unknown ?
winrt::box_value(winrt::to_hstring((int32_t)webErrorStatus)) :
winrt::box_value(winrt::to_hstring(ex.to_abi())));
        }
    }

    IAsyncAction
    ServerDatagramSocket_MessageReceived(Windows::Networking::Sockets::DatagramS
ocket sender,
Windows::Networking::Sockets::DatagramSocketMessageReceivedEventArgs args)
    {
        DataReader dataReader{ args.GetDataReader() };
    }

```

```

        winrt::hstring request{
dataReader.ReadString(dataReader.UnconsumedBufferLength()) };

serverListBox().Dispatcher().RunAsync(CoreDispatcherPriority::Normal, [=]()
{
    std::wstringstream wstringstream;
    wstringstream << L"server received the request: \"" <<
request.c_str() << L"\"";

serverListBox().Items().Append(winrt::box_value(wstringstream.str().c_str())
);

    });

    // Echo the request back as the response.
    IOutputStream outputStream = co_await
sender.GetOutputStreamAsync(args.RemoteAddress(), L"1336");
    DataWriter dataWriter{ outputStream };
    dataWriter.WriteString(request);

    co_await dataWriter.StoreAsync();
    dataWriter.DetachStream();

serverListBox().Dispatcher().RunAsync(CoreDispatcherPriority::Normal, [=]()
{
    std::wstringstream wstringstream;
    wstringstream << L"server sent back the response: \"" <<
request.c_str() << L"\"";

serverListBox().Items().Append(winrt::box_value(wstringstream.str().c_str())
);

    m_serverDatagramSocket = nullptr;

    serverListBox().Items().Append(winrt::box_value(L"server closed
its socket"));
    });
}

IAsyncAction StartClient()
{
    try
    {
        m_clientDatagramSocket.MessageReceived({ this,
&DatagramSocketPage::ClientDatagramSocket_MessageReceived });

        // Establish a connection to the echo server.
        // The server hostname that we will be establishing a connection
to. In this example, the server and client are in the same process.
        Windows::Networking::HostName hostName{ L"localhost" };

        clientListBox().Items().Append(winrt::box_value(L"client is
about to bind..."));
    }
}

```

```

        co_await m_clientDatagramSocket.BindServiceNameAsync(L"1336");
        clientListBox().Items().Append(winrt::box_value(L"client is
bound to port number 1336"));

        // Send a request to the echo server.
        Windows::Networking::Sockets::DatagramSocket
serverDatagramSocket;
        IOutputStream outputStream = co_await
m_serverDatagramSocket.GetOutputStreamAsync(hostName, L"1337");

        winrt::hstring request{ L"Hello, World!" };
        DataWriter dataWriter{ outputStream };
        dataWriter.WriteString(request);
        co_await dataWriter.StoreAsync();
        dataWriter.DetachStream();

        std::wstringstream wstringstream;
        wstringstream << L"client sent the request: \"" <<
request.c_str() << L"\n";

        clientListBox().Items().Append(winrt::box_value(wstringstream.str().c_str())
);
    }
    catch (winrt::hresult_error const& ex)
    {
        Windows::Networking::Sockets::SocketErrorStatus webErrorStatus{
Windows::Networking::Sockets::SocketError::GetStatus(ex.to_abi()) };
        serverListBox().Items().Append(webErrorStatus !=
Windows::Networking::Sockets::SocketErrorStatus::Unknown ?
winrt::box_value(winrt::to_hstring((int32_t)webErrorStatus)) :
winrt::box_value(winrt::to_hstring(ex.to_abi())));
    }
}

void
ClientDatagramSocket_MessageReceived(Windows::Networking::Sockets::DatagramS
ocket const& /* sender */,
Windows::Networking::Sockets::DatagramSocketMessageReceivedEventArgs const&
args)
{
    DataReader dataReader{ args.GetDataReader() };
    winrt::hstring response{
dataReader.ReadString(dataReader.UnconsumedBufferLength()) };

    clientListBox().Dispatcher().RunAsync(CoreDispatcherPriority::Normal, [=]()
    {
        std::wstringstream wstringstream;
        wstringstream << L"client received the response: \"" <<
response.c_str() << L"\n";

        clientListBox().Items().Append(winrt::box_value(wstringstream.str().c_str())
);
    });

    m_clientDatagramSocket = nullptr;

```

```

clientListBox().Dispatcher().RunAsync(CoreDispatcherPriority::Normal, [=]()
{
    clientListBox().Items().Append(winrt::box_value(L"client closed
its socket"));
});
}

```

C++

```

#include <ppltasks.h>
#include <sstream>

...
using namespace Windows::Foundation;
using namespace Windows::Storage::Streams;
using namespace Windows::UI::Core;
using namespace Windows::UI::Xaml::Navigation;
...
private:
    Windows::Networking::Sockets::DatagramSocket^ clientDatagramSocket;
    Windows::Networking::Sockets::DatagramSocket^ serverDatagramSocket;

protected:
    virtual void OnNavigatedTo(NavigationEventArgs^ e) override
    {
        this->StartServer();
        this->StartClient();
    }

private:
    void StartServer()
    {
        try
        {
            this->serverDatagramSocket = ref new
Windows::Networking::Sockets::DatagramSocket();

            // The ConnectionReceived event is raised when connections are
received.
            this->serverDatagramSocket->MessageReceived += ref new
TypedEventHandler<Windows::Networking::Sockets::DatagramSocket^,
Windows::Networking::Sockets::DatagramSocketMessageReceivedEventArgs^>(this,
&DatagramSocketPage::ServerDatagramSocket_MessageReceived);

            this->serverListBox->Items->Append(L"server is about to
bind...");

            // Start listening for incoming TCP connections on the specified
port. You can specify any port that's not currently in use.
            Concurrency::create_task(this->serverDatagramSocket-
>BindServiceNameAsync("1337")).then(
                [=]

```

```

        {
            this->serverListBox->Items->Append(L"server is bound to port
number 1337");
        });
    }
    catch (Platform::Exception^ ex)
    {
        Windows::Networking::Sockets::SocketErrorStatus webErrorStatus =
Windows::Networking::Sockets::SocketError::GetStatus(ex->HRESULT);
        this->serverListBox->Items->Append(webErrorStatus.ToString() !=
L"Unknown" ? webErrorStatus.ToString() : ex->Message);
    }
}

void
ServerDatagramSocket_MessageReceived(Windows::Networking::Sockets::DatagramS
ocket^ sender,
Windows::Networking::Sockets::DatagramSocketMessageReceivedEventArgs^ args)
{
    DataReader^ dataReader = args->GetDataReader();
    Platform::String^ request = dataReader->ReadString(dataReader-
>UnconsumedBufferLength);
    this->Dispatcher->RunAsync(CoreDispatcherPriority::Normal, ref new
DispatchedHandler(
        [=]
        {
            std::wstringstream wstringstream;
            wstringstream << L"server received the request: \"" << request-
>Data() << L"\\"";
            this->serverListBox->Items->Append(ref new
Platform::String(wstringstream.str().c_str()));
        }));

    // Echo the request back as the response.
    Concurrency::create_task(sender->GetOutputStreamAsync(args-
>RemoteAddress, "1336")).then(
        [=](IOutputStream^ outputStream)
        {
            auto dataWriter = ref new DataWriter(outputStream);
            dataWriter->WriteString(request);
            Concurrency::create_task(dataWriter->StoreAsync()).then(
                [=](unsigned int)
                {
                    dataWriter->DetachStream();

                    this->Dispatcher->RunAsync(CoreDispatcherPriority::Normal,
ref new DispatchedHandler(
                        [=]()
                        {
                            std::wstringstream wstringstream;
                            wstringstream << L"server sent back the response: \"" <<
request->Data() << L"\\"";
                            this->serverListBox->Items->Append(ref new
Platform::String(wstringstream.str().c_str()));

```



```

        std::wstringstream wstringstream;
        wstringstream << L"client sent the request: \"" <<
request->Data() << L"\\"";
        this->clientListBox->Items->Append(ref new
Platform::String(wstringstream.str().c_str()));
    });

}
catch (Platform::Exception^ ex)
{
    Windows::Networking::Sockets::SocketErrorStatus webErrorStatus =
Windows::Networking::Sockets::SocketError::GetStatus(ex->HRESULT);
    this->serverListBox->Items->Append(webErrorStatus.ToString() !=
L"Unknown" ? webErrorStatus.ToString() : ex->Message);
}
}

void
ClientDatagramSocket_MessageReceived(Windows::Networking::Sockets::DatagramS
ocket^ sender,
Windows::Networking::Sockets::DatagramSocketMessageReceivedEventArgs^ args)
{
    DataReader^ dataReader = args->GetDataReader();
    Platform::String^ response = dataReader->ReadString(dataReader-
>UnconsumedBufferLength);
    this->Dispatcher->RunAsync(CoreDispatcherPriority::Normal, ref new
DispatchedHandler(
        [=]
        {
            std::wstringstream wstringstream;
            wstringstream << L"client received the response: \"" <<
response->Data() << L"\\"";
            this->clientListBox->Items->Append(ref new
Platform::String(wstringstream.str().c_str()));
        }));

    delete this->clientDatagramSocket;
    this->clientDatagramSocket = nullptr;

    this->Dispatcher->RunAsync(CoreDispatcherPriority::Normal, ref new
DispatchedHandler([=]() {this->clientListBox->Items->Append(L"client closed
its socket"); }));
}

```

Background operations and the socket broker

You can use the socket broker, and control channel triggers, to ensure that your app properly receives connections or data on sockets while it's not in the foreground. For more info, see [Network communications in the background](#).

Batched sends

Whenever you write to the stream associated with a socket, a transition happens from user mode (your code) to kernel mode (where the network stack is). If you're writing many buffers at a time then these repeated transitions compound into substantial overhead. Batching your sends is a way to send multiple buffers of data together, and avoid that overhead. It's especially useful if your app is doing VoIP, VPN, or other tasks that involve moving a lot of data as efficiently as possible.

This section demonstrates a couple of batched sends techniques that you can use with a [StreamSocket](#) or a connected [DatagramSocket](#).

To get a baseline, let's see how to send a large number of buffers in an inefficient way. Here's a minimal demo, using a **StreamSocket**.

C#

```
protected override async void OnNavigatedTo(NavigationEventArgs e)
{
    var streamSocketListener = new
Windows.Networking.Sockets.StreamSocketListener();
    streamSocketListener.ConnectionReceived +=
this.StreamSocketListener_ConnectionReceived;
    await streamSocketListener.BindServiceNameAsync("1337");

    var streamSocket = new Windows.Networking.Sockets.StreamSocket();
    await streamSocket.ConnectAsync(new
Windows.Networking.HostName("localhost"), "1337");
    this.SendMultipleBuffersInefficiently(streamSocket, "Hello, World!");
    //this.BatchedSendsCSharpOnly(streamSocket, "Hello, World!");
    //this.BatchedSendsAnyUWPLanguage(streamSocket, "Hello, World!");
}

private async void
StreamSocketListener_ConnectionReceived(Windows.Networking.Sockets.StreamSocketListener sender,
Windows.Networking.Sockets.StreamSocketListenerConnectionReceivedEventArgs args)
{
    using (var dataReader = new DataReader(args.Socket.InputStream))
    {
        dataReader.InputStreamOptions = InputStreamOptions.Partial;
        while (true)
        {
            await dataReader.LoadAsync(256);
            if (dataReader.UnconsumedBufferLength == 0) break;
            IBuffer requestBuffer =
dataReader.ReadBuffer(dataReader.UnconsumedBufferLength);
            string request =
Windows.Security.Cryptography.CryptographicBuffer.ConvertBinaryToString(Wind
```

```

ows.Security.Cryptography.BinaryStringEncoding.Utf8, requestBuffer);
    Debug.WriteLine(string.Format("server received the request: \"{0}\"", request));
}
}
}

// This implementation incurs kernel transition overhead for each packet
// written.
private async void
SendMultipleBuffersInefficiently(Windows.Networking.Sockets.StreamSocket
streamSocket, string message)
{
    var packetsToSend = new List<IBuffer>();
    for (int count = 0; count < 5; ++count) {
        packetsToSend.Add(Windows.Security.Cryptography.CryptographicBuffer.ConvertS
tringToBinary(message,
Windows.Security.Cryptography.BinaryStringEncoding.Utf8)); }

    foreach (IBuffer packet in packetsToSend)
    {
        await streamSocket.OutputStream.WriteAsync(packet);
    }
}

```

C++/WinRT

```

#include <winrt/Windows.Foundation.h>
#include <winrt/Windows.Networking.Sockets.h>
#include <winrt/Windows.Security.Cryptography.h>
#include <winrt/Windows.Storage.Streams.h>
#include <winrt/Windows.UI.Core.h>
#include <winrt/Windows.UI.Xaml.Navigation.h>
#include <sstream>

using namespace winrt;
using namespace Windows::Foundation;
using namespace Windows::Storage::Streams;
using namespace Windows::UI::Core;
using namespace Windows::UI::Xaml::Navigation;
...
private:
    Windows::Networking::Sockets::StreamSocketListener
m_streamSocketListener;
    Windows::Networking::Sockets::StreamSocket m_streamSocket;

public:
    IAsyncAction OnNavigatedTo(NavigationEventArgs /* e */)
    {
        m_streamSocketListener.ConnectionReceived({ this,
&BatchedSendsPage::OnConnectionReceived });
        co_await m_streamSocketListener.BindServiceNameAsync(L"1337");
    }
}

```

```

        co_await m_streamSocket.ConnectAsync(Windows::Networking::HostName{
L"localhost" }, L"1337");
        SendMultipleBuffersInefficientlyAsync(L"Hello, World!");
        //BatchedSendsAnyUWPLanguageAsync(L"Hello, World!");
    }

private:
    IAsyncAction
OnConnectionReceived(Windows::Networking::Sockets::StreamSocketListener
const& /* sender */,
Windows::Networking::Sockets::StreamSocketListenerConnectionReceivedEventArgs const& args)
    {
        DataReader dataReader{ args.Socket().InputStream() };

dataReader.InputStreamOptions(Windows::Storage::Streams::InputStreamOptions:
:Partial);

        while (true)
        {
            unsigned int bytesLoaded = co_await dataReader.LoadAsync(256);
            if (bytesLoaded == 0) break;
            winrt::hstring message{ dataReader.ReadString(bytesLoaded) };
            ::OutputDebugString(message.c_str());
        }
    }

    // This implementation incurs kernel transition overhead for each packet
    written.
    IAsyncAction SendMultipleBuffersInefficientlyAsync(winrt::hstring
message)
    {
        co_await winrt::resume_background();

        std::vector< IBuffer > packetsToSend;
        for (unsigned int count = 0; count < 5; ++count)
        {

packetsToSend.push_back(Windows::Security::Cryptography::CryptographicBuffer
::ConvertStringToBinary(message,
Windows::Security::Cryptography::BinaryStringEncoding::Utf8));
        }

        for (auto const& element : packetsToSend)
        {
            m_streamSocket.OutputStream().WriteAsync(element).get();
        }
    }
}

```

C++

```

#include <ppltasks.h>
#include <sstream>

```

```

...
using namespace Windows::Foundation;
using namespace Windows::Storage::Streams;
using namespace Windows::UI::Core;
using namespace Windows::UI::Xaml::Navigation;
...
private:
    Windows::Networking::Sockets::StreamSocketListener^
streamSocketListener;
    Windows::Networking::Sockets::StreamSocket^ streamSocket;

protected:
    virtual void OnNavigatedTo(NavigationEventArgs^ e) override
    {
        this->streamSocketListener = ref new
Windows::Networking::Sockets::StreamSocketListener();
        streamSocketListener->ConnectionReceived += ref new
TypedEventHandler<Windows::Networking::Sockets::StreamSocketListener^,
Windows::Networking::Sockets::StreamSocketListenerConnectionReceivedEventArg
s^>(this, &BatchedSendsPage::StreamSocketListener_ConnectionReceived);
        Concurrency::create_task(this->streamSocketListener->
>BindServiceNameAsync(L"1337")).then(
            [=]
            {
                this->streamSocket = ref new
Windows::Networking::Sockets::StreamSocket();
                Concurrency::create_task(this->streamSocket->ConnectAsync(ref
new Windows::Networking::HostName(L"localhost"), L"1337")).then(
                    [=](Concurrency::task< void >)
                    {
                        this->SendMultipleBuffersInefficiently(L"Hello, World!");
                        // this->BatchedSendsAnyUWPLanguage(L"Hello, World!");
                    },
                );
                Concurrency::task_continuation_context::use_synchronous_execution();
            });
    }

private:
    void
StreamSocketListener_ConnectionReceived(Windows::Networking::Sockets::Stream
SocketListener^ sender,
Windows::Networking::Sockets::StreamSocketListenerConnectionReceivedEventArg
s^ args)
    {
        auto dataReader = ref new DataReader(args->Socket->InputStream);
        dataReader->InputStreamOptions =
Windows::Storage::Streams::InputStreamOptions::Partial;
        this->ReceiveStringRecurse(dataReader, args->Socket);
    }

    void ReceiveStringRecurse(DataReader^ dataReader,
Windows::Networking::Sockets::StreamSocket^ streamSocket)
    {
        Concurrency::create_task(dataReader->LoadAsync(256)).then(
            [this, dataReader, streamSocket](unsigned int bytesLoaded)

```

```

    {
        if (bytesLoaded == 0) return;
        Platform::String^ message = dataReader->ReadString(bytesLoaded);
        ::OutputDebugString(message->Data());
        this->ReceiveStringRecurse(dataReader, streamSocket);
    });
}

// This implementation incurs kernel transition overhead for each packet
// written.
void SendMultipleBuffersInefficiently(Platform::String^ message)
{
    std::vector< IBuffer^ > packetsToSend{};
    for (unsigned int count = 0; count < 5; ++count)
    {

packetsToSend.push_back(Windows::Security::Cryptography::CryptographicBuffer
::ConvertStringToBinary(message,
Windows::Security::Cryptography::BinaryStringEncoding::Utf8));
    }

    for (auto element : packetsToSend)
    {
        Concurrency::create_task(this->streamSocket->OutputStream-
>WriteAsync(element)).wait();
    }
}

```

This first example of a more efficient technique is only appropriate if you're using C#. Change `OnNavigatedTo` to call `BatchedSendsCSharpOnly` instead of `SendMultipleBuffersInefficiently` or `SendMultipleBuffersInefficientlyAsync`.

C#

```

// A C#-only technique for batched sends.
private async void
BatchedSendsCSharpOnly(Windows.Networking.Sockets.StreamSocket streamSocket,
string message)
{
    var packetsToSend = new List<IBuffer>();
    for (int count = 0; count < 5; ++count) {
packetsToSend.Add(Windows.Security.Cryptography.CryptographicBuffer.ConvertS
tringToBinary(message,
Windows.Security.Cryptography.BinaryStringEncoding.Utf8)); }

    var pendingTasks = new System.Threading.Tasks.Task[packetsToSend.Count];

    for (int index = 0; index < packetsToSend.Count; ++index)
    {
        // track all pending writes as tasks, but don't wait on one before
        // beginning the next.
        pendingTasks[index] =

```

```

streamSocket.OutputStream.WriteAsync(packetsToSend[index]).AsTask();
    // Don't modify any buffer's contents until the pending writes are
    complete.
}

// Wait for all of the pending writes to complete.
System.Threading.Tasks.Task.WaitAll(pendingTasks);
}

```

This next example is appropriate for any UWP language, not just for C#. It relies on the behavior in [StreamSocket.OutputStream](#) and [DatagramSocket.OutputStream](#) that batches sends together. The technique calls [FlushAsync](#) on that output stream which, as of Windows 10, is guaranteed to return only after all operations on the output stream have completed.

C#

```

// An implementation of batched sends suitable for any UWP language.
private async void
BatchedSendsAnyUWPLanguage(Windows.Networking.Sockets.StreamSocket
streamSocket, string message)
{
    var packetsToSend = new List<IBuffer>();
    for (int count = 0; count < 5; ++count) {
        packetsToSend.Add(Windows.Security.Cryptography.CryptographicBuffer.ConvertS
tringToBinary(message,
Windows.Security.Cryptography.BinaryStringEncoding.Utf8)); }

    var pendingWrites = new IAsyncOperationWithProgress<uint, uint>
[packetsToSend.Count];

    for (int index = 0; index < packetsToSend.Count; ++index)
    {
        // track all pending writes as tasks, but don't wait on one before
        beginning the next.
        pendingWrites[index] =
streamSocket.OutputStream.WriteAsync(packetsToSend[index]);
        // Don't modify any buffer's contents until the pending writes are
        complete.
    }

    // Wait for all of the pending writes to complete. This step enables
    batched sends on the output stream.
    await streamSocket.OutputStream.FlushAsync();
}

```

C++/WinRT

```

// An implementation of batched sends suitable for any UWP language.
IAsyncAction BatchedSendsAnyUWPLanguageAsync(winrt::hstring message)

```

```

{
    std::vector< IBuffer > packetsToSend{};
    std::vector< IAsyncOperationWithProgress< unsigned int, unsigned int > >
pendingWrites{};
    for (unsigned int count = 0; count < 5; ++count)
    {

packetsToSend.push_back(Windows::Security::Cryptography::CryptographicBuffer
::ConvertStringToBinary(message,
Windows::Security::Cryptography::BinaryStringEncoding::Utf8));
    }

    for (auto const& element : packetsToSend)
    {
        // track all pending writes as tasks, but don't wait on one before
beginning the next.

pendingWrites.push_back(m_streamSocket.OutputStream().WriteAsync(element));
        // Don't modify any buffer's contents until the pending writes are
complete.
    }

    // Wait for all of the pending writes to complete. This step enables
batched sends on the output stream.
    co_await m_streamSocket.OutputStream().FlushAsync();
}

```

C++

```

private:
    // An implementation of batched sends suitable for any UWP language.
    void BatchedSendsAnyUWPLanguage(Platform::String^ message)
    {
        std::vector< IBuffer^ > packetsToSend{};
        std::vector< IAsyncOperationWithProgress< unsigned int, unsigned int
>^ >pendingWrites{};
        for (unsigned int count = 0; count < 5; ++count)
        {

packetsToSend.push_back(Windows::Security::Cryptography::CryptographicBuffer
::ConvertStringToBinary(message,
Windows::Security::Cryptography::BinaryStringEncoding::Utf8));
        }

        for (auto element : packetsToSend)
        {
            // track all pending writes as tasks, but don't wait on one
before beginning the next.
            pendingWrites.push_back(this->streamSocket->OutputStream-
>WriteAsync(element));
            // Don't modify any buffer's contents until the pending writes
are complete.
        }
    }

```

```
// Wait for all of the pending writes to complete. This step enables
batched sends on the output stream.
Concurrency::create_task(this->streamSocket->OutputStream-
>FlushAsync());
}
```

There are some important limitations imposed by using batched sends in your code.

- You cannot modify the contents of the **IBuffer** instances being written until the asynchronous write is complete.
- The **FlushAsync** pattern only works on **StreamSocket.OutputStream** and **DatagramSocket.OutputStream**.
- The **FlushAsync** pattern only works in Windows 10 and onward.
- In other cases, use **Task.WaitAll** instead of the **FlushAsync** pattern.

Port sharing for DatagramSocket

You can configure a **DatagramSocket** to coexist with other Win32 or UWP multicast sockets bound to the same address/port. You do this by setting the **DatagramSocketControl.MulticastOnly** to `true` before binding or connecting the socket. You access an instance of **DatagramSocketControl** from the **DatagramSocket** object itself via its **DatagramSocket.Control** property.

Providing a client certificate with the StreamSocket class

StreamSocket supports using SSL/TLS to authenticate the server that the client app is talking to. In some cases, the client app needs to authenticate itself to the server using an SSL/TLS client certificate. You can provide a client certificate with the **StreamSocketControl.ClientCertificate** property before binding or connecting the socket (it must be set before the SSL/TLS handshake is started). You access an instance of **StreamSocketControl** from the **StreamSocket** object itself via its **StreamSocket.Control** property. If the server requests the client certificate then Windows will respond with the client certificate that you provided.

Use an override of **StreamSocket.ConnectAsync** that takes a **SocketProtectionLevel**, as shown in this minimal code example.

 **Important**

As indicated by the comment in the code examples below, your project needs to declare the sharedUserCertificates app capability for this code to work.

C#

```
// For this code to work, you need at least one certificate to be present in
// the user MY certificate store.
// Plugging a smartcard into a smartcard reader connected to your PC will
// achieve that.
// Also, your project needs to declare the sharedUserCertificates app
// capability.
var certificateQuery = new
Windows.Security.Cryptography.Certificates.CertificateQuery();
certificateQuery.StoreName = "MY";
IReadOnlyList<Windows.Security.Cryptography.Certificates.Certificate>
certificates = await
Windows.Security.Cryptography.Certificates.CertificateStores.FindAllAsync(certificateQuery);
if (certificates.Count > 0)
{
    streamSocket.Control.ClientCertificate = certificates[0];
    await streamSocket.ConnectAsync(hostName, "1337",
Windows.Networking.Sockets.SocketProtectionLevel.Tls12);
}
```

C++/WinRT

```
// For this code to work, you need at least one certificate to be present in
// the user MY certificate store.
// Plugging a smartcard into a smartcard reader connected to your PC will
// achieve that.
// Also, your project needs to declare the sharedUserCertificates app
// capability.
Windows::Security::Cryptography::Certificates::CertificateQuery
certificateQuery;
certificateQuery.StoreName(L"MY");
IVectorView< Windows::Security::Cryptography::Certificates::Certificate >
certificates = co_await
Windows::Security::Cryptography::Certificates::CertificateStores::FindAllAsync(certificateQuery);

if (certificates.Size() > 0)
{
    m_streamSocket.Control().ClientCertificate(certificates.GetAt(0));
    co_await m_streamSocket.ConnectAsync(Windows::Networking::HostName{
L"localhost" }, L"1337",
Windows::Networking::Sockets::SocketProtectionLevel::Tls12);
    ...
}
```

C++

```
// For this code to work, you need at least one certificate to be present in
// the user MY certificate store.
// Plugging a smartcard into a smartcard reader connected to your PC will
// achieve that.
// Also, your project needs to declare the sharedUserCertificates app
// capability.
auto certificateQuery = ref new
Windows::Security::Cryptography::Certificates::CertificateQuery();
certificateQuery->StoreName = L"MY";
Concurrency::create_task(Windows::Security::Cryptography::Certificates::CertificateStores::FindAllAsync(certificateQuery)).then(
    [=](IVectorView<
Windows::Security::Cryptography::Certificates::Certificate^ > certificates)
    {
        if (certificates->Size > 0)
        {
            this->streamSocket->Control->ClientCertificate = certificates-
>GetAt(0);
            Concurrency::create_task(this->streamSocket->ConnectAsync(ref new
Windows::Networking::HostName(L"localhost"), L"1337",
Windows::Networking::Sockets::SocketProtectionLevel::Tls12)).then(
                [=]
                {
                    ...
                });
        }
    });
});
```

Handling exceptions

An error encountered on a [DatagramSocket](#), [StreamSocket](#), or [StreamSocketListener](#) operation is returned as an **HRESULT** value. You can pass that **HRESULT** value to the [SocketError.GetStatus](#) method to convert it into a [SocketErrorStatus](#) enumeration value.

Most [SocketErrorStatus](#) enumeration values correspond to an error returned by the native Windows sockets operation. Your app can switch on [SocketErrorStatus](#) enumeration values to modify app behavior depending on the cause of the exception.

For parameter validation errors, you can use the **HRESULT** from the exception to learn more detailed information about the error. Possible **HRESULT** values are listed in `Winerror.h`, which can be found in your SDK installation (for example, in the folder `C:\Program Files (x86)\Windows Kits\10\Include\<VERSION>\shared`). For most parameter validation errors, the **HRESULT** returned is **E_INVALIDARG**.

The [HostName](#) constructor can throw an exception if the string passed is not a valid host name. For example, it contains characters that are not allowed, which is likely if the

host name is typed in to your app by the user. Construct a **HostName** inside a try/catch block. That way, if an exception is thrown, the app can notify the user and request a new host name.

Important APIs

- [CertificateQuery](#)
- [CertificateStores.FindAllAsync](#)
- [DatagramSocket](#)
- [DatagramSocket.BindServiceNameAsync](#)
- [DatagramSocket.Control](#)
- [DatagramSocket.GetOutputStreamAsync](#)
- [DatagramSocket.MessageReceived](#)
- [DatagramSocketControl.MulticastOnly](#)
- [DatagramSocketMessageReceivedEventArgs](#)
- [DatagramSocketMessageReceivedEventArgs.GetDataReader](#)
- [DataReader.LoadAsync](#)
- [IOutputStream.FlushAsync](#)
- [SocketError.GetStatus](#)
- [SocketErrorStatus](#)
- [SocketProtectionLevel](#)
- [StreamSocket](#)
- [StreamSocketControl.ClientCertificate](#)
- [StreamSocket.ConnectAsync](#)
- [StreamSocket.InputStream](#)
- [StreamSocket.OutputStream](#)
- [StreamSocketListener](#)
- [StreamSocketListener.BindServiceNameAsync](#)
- [StreamSocketListener.ConnectionReceived](#)
- [StreamSocketListenerConnectionReceivedEventArgs](#)
- [Windows.Networking.Sockets](#)

Related topics

- [App-to-app communication](#)
- [Concurrency and asynchronous operations with C++/WinRT](#)
- [How to set network capabilities](#)
- [Windows Sockets 2 \(Winsock\)](#)

Samples

- [StreamSocket sample](#) 

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

WebSockets

Article • 10/20/2022

WebSockets provide a mechanism for fast, secure, two-way communication between a client and a server over the web using HTTP(S), and supporting both UTF-8 and binary messages.

Under the [WebSocket Protocol](#), data is transferred immediately over a full-duplex single socket connection, allowing messages to be sent and received from both endpoints in real time. WebSockets are ideal for use in multiplayer gaming (both real-time and turn-based), instant social network notifications, up-to-date displays of stock or weather information, and other apps requiring secure and fast data transfer.

To establish a WebSocket connection, a specific, HTTP-based handshake is exchanged between the client and the server. If successful, the application-layer protocol is "upgraded" from HTTP to WebSockets, using the previously established TCP connection. Once this occurs, HTTP is completely out of the picture; data can be sent or received using the WebSocket protocol by both endpoints, until the WebSocket connection is closed.

Note A client cannot use WebSockets to transfer data unless the server also uses the WebSocket protocol. If the server does not support WebSockets, then you must use another method of data transfer.

The Universal Windows Platform (UWP) provides support for both client and server use of WebSockets. The [Windows.Networking.Sockets](#) namespace defines two WebSocket classes for use by clients—[MessageWebSocket](#), and [StreamWebSocket](#). Here's a comparison of these two WebSocket classes.

MessageWebSocket	StreamWebSocket
An entire WebSocket message is read/written in a single operation.	Sections of a message can be read with each read operation.
Suitable when messages are not very large.	Suitable when very large files (such as photos or videos) are being transferred.
Supports both UTF-8 and binary messages.	Supports only binary messages.

MessageWebSocket	StreamWebSocket
Similar to a UDP or datagram socket (in the sense of being intended for frequent, small messages), but with TCP's reliability, packet order guarantees, and congestion control.	Similar to a TCP or stream socket .

Secure your connection with TLS/SSL

In most cases, you'll want to use a secure WebSocket connection so that the data you send and receive is encrypted. This will also increase the chances that your connection will succeed, because many intermediaries such as firewalls and proxies reject unencrypted WebSocket connections. The [WebSocket protocol](#) [↗] defines these two URI schemes.

URI scheme	Purpose
wss:	Use for secure connections that should be encrypted.
ws:	Use for unencrypted connections.

To encrypt your WebSocket connection, use the `wss:` URI scheme. Here's an example.

C#

```
protected override async void OnNavigatedTo(NavigationEventArgs e)
{
    var websocket = new Windows.Networking.Sockets.MessageWebSocket();
    await websocket.ConnectAsync(new
Uri("wss://www.contoso.com/mywebservice"));
}
```

C++/WinRT

```
#include <winrt/Windows.Foundation.h>
#include <winrt/Windows.Networking.Sockets.h>
#include <winrt/Windows.UI.Xaml.Navigation.h>
#include <sstream>

using namespace winrt;
using namespace Windows::Foundation;
using namespace Windows::UI::Xaml::Navigation;
...
IAsyncAction OnNavigatedTo(NavigationEventArgs /* e */)
{
    Windows::Networking::Sockets::MessageWebSocket websocket;
    co_await websocket.ConnectAsync(Uri{
```

```
L"wss://www.contoso.com/mywebservice" });  
}
```

Use MessageWebSocket to connect

MessageWebSocket allows an entire WebSocket message to be read/written in a single operation. Consequently, it's suitable when messages are not very large. The class supports both UTF-8 and binary messages.

The example code below uses the WebSocket.org echo server—a service that echoes back to the sender any message sent to it.

C#

```
private Windows.Networking.Sockets.MessageWebSocket messageWebSocket;  
  
protected override void OnNavigatedTo(NavigationEventArgs e)  
{  
    this.messageWebSocket = new  
Windows.Networking.Sockets.MessageWebSocket();  
  
    // In this example, we send/receive a string, so we need to set the  
MessageType to Utf8.  
    this.messageWebSocket.Control.MessageType =  
Windows.Networking.Sockets.SocketMessageType.Utf8;  
  
    this.messageWebSocket.MessageReceived += WebSocket_MessageReceived;  
    this.messageWebSocket.Closed += WebSocket_Closed;  
  
    try  
    {  
        Task connectTask = this.messageWebSocket.ConnectAsync(new  
Uri("wss://echo.websocket.org")).AsTask();  
        connectTask.ContinueWith(_ =>  
this.SendMessageUsingMessageWebSocketAsync("Hello, World!"));  
    }  
    catch (Exception ex)  
    {  
        Windows.Web.WebErrorStatus webErrorStatus =  
Windows.Networking.Sockets.WebSocketError.GetStatus(ex.GetBaseException().HR  
esult);  
        // Add additional code here to handle exceptions.  
    }  
}  
  
private async Task SendMessageUsingMessageWebSocketAsync(string message)  
{  
    using (var dataWriter = new  
DataWriter(this.messageWebSocket.OutputStream))  
    {
```

```

        dataWriter.WriteString(message);
        await dataWriter.StoreAsync();
        dataWriter.DetachStream();
    }
    Debug.WriteLine("Sending message using MessageWebSocket: " + message);
}

private void
WebSocket_MessageReceived(Windows.Networking.Sockets.MessageWebSocket
sender, Windows.Networking.Sockets.MessageWebSocketMessageReceivedEventArgs
args)
{
    try
    {
        using (DataReader dataReader = args.GetDataReader())
        {
            dataReader.UnicodeEncoding =
Windows.Storage.Streams.UnicodeEncoding.Utf8;
            string message =
dataReader.ReadString(dataReader.UnconsumedBufferLength);
            Debug.WriteLine("Message received from MessageWebSocket: " +
message);
            this.messageWebSocket.Dispose();
        }
    }
    catch (Exception ex)
    {
        Windows.Web.WebErrorStatus webErrorStatus =
Windows.Networking.Sockets.WebSocketError.GetStatus(ex.GetBaseException().HR
esult);
        // Add additional code here to handle exceptions.
    }
}

private void WebSocket_Closed(Windows.Networking.Sockets.IWebSocket sender,
Windows.Networking.Sockets.WebSocketClosedEventArgs args)
{
    Debug.WriteLine("WebSocket_Closed; Code: " + args.Code + ", Reason: \""
+ args.Reason + "\"");
    // Add additional code here to handle the WebSocket being closed.
}

```

C++/WinRT

```

#include <winrt/Windows.Foundation.h>
#include <winrt/Windows.Networking.Sockets.h>
#include <winrt/Windows.Storage.Streams.h>
#include <winrt/Windows.UI.Xaml.Navigation.h>
#include <sstream>

using namespace winrt;
using namespace Windows::Foundation;
using namespace Windows::Storage::Streams;

```

```

using namespace Windows::UI::Xaml::Navigation;
...
private:
    Windows::Networking::Sockets::MessageWebSocket m_messageWebSocket;
    winrt::event_token m_messageReceivedEventToken;
    winrt::event_token m_closedEventToken;

public:
    IAsyncAction OnNavigatedTo(NavigationEventArgs /* e */)
    {
        // In this example, we send/receive a string, so we need to set the
        MessageType to Utf8.

        m_messageWebSocket.Control().MessageType(Windows::Networking::Sockets::SocketMessageType::Utf8);

        m_messageReceivedEventToken = m_messageWebSocket.MessageReceived({
            this, &MessageWebSocketPage::OnWebSocketMessageReceived });
        m_closedEventToken = m_messageWebSocket.Closed({ this,
            &MessageWebSocketPage::OnWebSocketClosed });

        try
        {
            co_await m_messageWebSocket.ConnectAsync(Uri{
                L"wss://echo.websocket.org" });
            SendMessageUsingMessageWebSocketAsync(L"Hello, World!");
        }
        catch (winrt::hresult_error const& ex)
        {
            Windows::Web::WebErrorStatus webErrorStatus{
                Windows::Networking::Sockets::WebSocketError::GetStatus(ex.to_abi()) };
            // Add additional code here to handle exceptions.
        }
    }

private:
    IAsyncAction SendMessageUsingMessageWebSocketAsync(std::wstring message)
    {
        DataWriter dataWriter{ m_messageWebSocket.OutputStream() };
        dataWriter.WriteString(message);

        co_await dataWriter.StoreAsync();
        dataWriter.DetachStream();
        std::wstringstream wstringstream;
        wstringstream << L"Sending message using MessageWebSocket: " <<
            message.c_str() << std::endl;
        ::OutputDebugString(wstringstream.str().c_str());
    }

    void
    OnWebSocketMessageReceived(Windows::Networking::Sockets::MessageWebSocket
        const& /* sender */,
        Windows::Networking::Sockets::MessageWebSocketMessageReceivedEventArgs
        const& args)
    {

```

```

try
{
    DataReader dataReader{ args.GetDataReader() };

    dataReader.UnicodeEncoding(Windows::Storage::Streams::UnicodeEncoding::Utf8)
;

    auto message =
dataReader.ReadString(dataReader.UnconsumedBufferLength());
    std::wstringstream wstringstream;
    wstringstream << L"Message received from MessageWebSocket: " <<
message.c_str() << std::endl;
    ::OutputDebugString(wstringstream.str().c_str());
    m_messageWebSocket.Close(1000, L"");
}
catch (winrt::hresult_error const& ex)
{
    Windows::Web::WebErrorStatus webErrorStatus{
Windows::Networking::Sockets::WebSocketError::GetStatus(ex.to_abi()) };
    // Add additional code here to handle exceptions.
}
}

void OnWebSocketClosed(Windows::Networking::Sockets::IWebSocket const&
/* sender */, Windows::Networking::Sockets::WebSocketClosedEventArgs const&
args)
{
    std::wstringstream wstringstream;
    wstringstream << L"WebSocket_Closed; Code: " << args.Code() << ",
Reason: \"" << args.Reason().c_str() << "\"" << std::endl;
    ::OutputDebugString(wstringstream.str().c_str());
    // Add additional code here to handle the WebSocket being closed.
}
}

```

C++

```

#include <ppltasks.h>
#include <sstream>
...
using namespace Windows::Foundation;
using namespace Windows::Storage::Streams;
using namespace Windows::UI::Xaml::Navigation;
...
private:
    Windows::Networking::Sockets::MessageWebSocket^ messageWebSocket;

protected:
    virtual void OnNavigatedTo(NavigationEventArgs^ e) override
    {
        this->messageWebSocket = ref new
Windows::Networking::Sockets::MessageWebSocket();

        // In this example, we send/receive a string, so we need to set the

```

MessageType to Utf8.

```
    this->messageWebSocket->Control->MessageType =
Windows::Networking::Sockets::SocketMessageType::Utf8;

    this->messageWebSocket->MessageReceived += ref new
TypedEventHandler<Windows::Networking::Sockets::MessageWebSocket^,
Windows::Networking::Sockets::MessageWebSocketMessageReceivedEventArgs^>
(this, &MessageWebSocketPage::WebSocket_MessageReceived);
    this->messageWebSocket->Closed += ref new
TypedEventHandler<Windows::Networking::Sockets::IWebSocket^,
Windows::Networking::Sockets::WebSocketClosedEventArgs^>(this,
&MessageWebSocketPage::WebSocket_Closed);

    try
    {
        auto connectTask = Concurrency::create_task(this-
>messageWebSocket->ConnectAsync(ref new Uri(L"wss://echo.websocket.org")));
        connectTask.then([this] { this-
>SendMessageUsingMessageWebSocketAsync(L"Hello, World!"); });
    }
    catch (Platform::Exception^ ex)
    {
        Windows::Web::WebErrorStatus webErrorStatus =
Windows::Networking::Sockets::WebSocketError::GetStatus(ex->HResult);
        // Add additional code here to handle exceptions.
    }
}

private:
void SendMessageUsingMessageWebSocketAsync(Platform::String^ message)
{
    auto dataWriter = ref new DataWriter(this->messageWebSocket-
>OutputStream);
    dataWriter->WriteString(message);

    Concurrency::create_task(dataWriter->StoreAsync()).then(
[=](unsigned int)
{
    dataWriter->DetachStream();
    std::wstringstream wstringstream;
    wstringstream << L"Sending message using MessageWebSocket: " <<
message->Data() << std::endl;
    ::OutputDebugString(wstringstream.str().c_str());
});
}

void
WebSocket_MessageReceived(Windows::Networking::Sockets::MessageWebSocket^
sender,
Windows::Networking::Sockets::MessageWebSocketMessageReceivedEventArgs^
args)
{
    try
    {
        DataReader^ dataReader = args->GetDataReader();
```

```

        dataReader->UnicodeEncoding =
Windows::Storage::Streams::UnicodeEncoding::Utf8;
        Platform::String^ message = dataReader->ReadString(dataReader-
>UnconsumedBufferLength);
        std::wstringstream wstringstream;
        wstringstream << L"Message received from MessageWebSocket: " <<
message->Data() << std::endl;
        ::OutputDebugString(wstringstream.str().c_str());
        this->messageWebSocket->Close(1000, L"");
    }
    catch (Platform::Exception^ ex)
    {
        Windows::Web::WebErrorStatus webErrorStatus =
Windows::Networking::Sockets::WebSocketError::GetStatus(ex->HResult);
        // Add additional code here to handle exceptions.
    }
}

void WebSocket_Closed(Windows::Networking::Sockets::IWebSocket^ sender,
Windows::Networking::Sockets::WebSocketClosedEventArgs^ args)
{
    std::wstringstream wstringstream;
    wstringstream << L"WebSocket_Closed; Code: " << args->Code << ",
Reason: \"" << args->Reason->Data() << "\"\" << std::endl;
    ::OutputDebugString(wstringstream.str().c_str());
    // Add additional code here to handle the WebSocket being closed.
}

```

Handle the MessageWebSocket.MessageReceived and MessageWebSocket.Closed events

As shown in the example above, before establishing a connection and sending data with a **MessageWebSocket**, you should subscribe to the

MessageWebSocket.MessageReceived and **MessageWebSocket.Closed** events.

MessageReceived is raised when data is received. The data can be accessed via **MessageWebSocketMessageReceivedEventArgs**. **Closed** is raised when the client or the server closes the socket.

Send data on a MessageWebSocket

Once a connection is established, you can send data to the server. You do this by using the **MessageWebSocket.OutputStream** property, and a **DataWriter**, to write the data.

Note The **DataWriter** takes ownership of the output stream. When the **DataWriter** goes out of scope, if the output stream is attached to it, the **DataWriter** deallocates the

output stream. After that, subsequent attempts to use the output stream fail with an HRESULT value of 0x80000013. But you can call [DataWriter.DetachStream](#) to detach the output stream from the **DataWriter** and return ownership of the stream to the **MessageWebSocket**.

Use StreamWebSocket to connect

[StreamWebSocket](#) allows sections of a message to be read with each read operation. Consequently, it's suitable when very large files (such as photos or videos) are being transferred. The class supports only binary messages.

The example code below uses the WebSocket.org echo server—a service that echoes back to the sender any message sent to it.

C#

```
private Windows.Networking.Sockets.StreamWebSocket streamWebSocket;

protected override void OnNavigatedTo(NavigationEventArgs e)
{
    this.streamWebSocket = new Windows.Networking.Sockets.StreamWebSocket();

    this.streamWebSocket.Closed += WebSocket_Closed;

    try
    {
        Task connectTask = this.streamWebSocket.ConnectAsync(new
Uri("wss://echo.websocket.org")).AsTask();

        connectTask.ContinueWith(_ =>
        {
            Task.Run(() => this.ReceiveMessageUsingStreamWebSocket());
            Task.Run(() => this.SendMessageUsingStreamWebSocket(new byte[] {
0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09 }));
        });
    }
    catch (Exception ex)
    {
        Windows.Web.WebErrorStatus webErrorStatus =
Windows.Networking.Sockets.WebSocketError.GetStatus(ex.GetBaseException().HR
esult);
        // Add code here to handle exceptions.
    }
}

private async void ReceiveMessageUsingStreamWebSocket()
{
    try
    {
        using (var dataReader = new
```

```

DataReader(this.streamWebSocket.InputStream))
{
    dataReader.InputStreamOptions = InputStreamOptions.Partial;
    await dataReader.LoadAsync(256);
    byte[] message = new byte[dataReader.UnconsumedBufferLength];
    dataReader.ReadBytes(message);
    Debug.WriteLine("Data received from StreamWebSocket: " +
message.Length + " bytes");
}
this.streamWebSocket.Dispose();
}
catch (Exception ex)
{
    Windows.Web.WebErrorStatus webErrorStatus =
Windows.Networking.Sockets.WebSocketError.GetStatus(ex.GetBaseException().HR
esult);
    // Add code here to handle exceptions.
}
}

private async void SendMessageUsingStreamWebSocket(byte[] message)
{
    try
    {
        using (var dataWriter = new
DataWriter(this.streamWebSocket.OutputStream))
        {
            dataWriter.WriteBytes(message);
            await dataWriter.StoreAsync();
            dataWriter.DetachStream();
        }
        Debug.WriteLine("Sending data using StreamWebSocket: " +
message.Length.ToString() + " bytes");
    }
    catch (Exception ex)
    {
        Windows.Web.WebErrorStatus webErrorStatus =
Windows.Networking.Sockets.WebSocketError.GetStatus(ex.GetBaseException().HR
esult);
        // Add code here to handle exceptions.
    }
}

private void WebSocket_Closed(Windows.Networking.Sockets.IWebSocket sender,
Windows.Networking.Sockets.WebSocketClosedEventArgs args)
{
    Debug.WriteLine("WebSocket_Closed; Code: " + args.Code + ", Reason: \""
+ args.Reason + "\"");
    // Add additional code here to handle the WebSocket being closed.
}

```

```

#include <winrt/Windows.Foundation.h>
#include <winrt/Windows.Networking.Sockets.h>
#include <winrt/Windows.Storage.Streams.h>
#include <winrt/Windows.UI.Xaml.Navigation.h>
#include <sstream>

using namespace winrt;
using namespace Windows::Foundation;
using namespace Windows::Storage::Streams;
using namespace Windows::UI::Xaml::Navigation;
...
private:
    Windows::Networking::Sockets::StreamWebSocket m_streamWebSocket;
    winrt::event_token m_closedEventToken;

public:
    IAsyncAction OnNavigatedTo(NavigationEventArgs /* e */)
    {
        m_closedEventToken = m_streamWebSocket.Closed({ this,
&StreamWebSocketPage::OnWebSocketClosed });

        try
        {
            co_await m_streamWebSocket.ConnectAsync(Uri{
L"wss://echo.websocket.org" });
            ReceiveMessageUsingStreamWebSocket();
            SendMessageUsingStreamWebSocket({ 0x00, 0x01, 0x02, 0x03, 0x04,
0x05, 0x06, 0x07, 0x08, 0x09 });
        }
        catch (winrt::hresult_error const& ex)
        {
            Windows::Web::WebErrorStatus webErrorStatus{
Windows::Networking::Sockets::WebSocketError::GetStatus(ex.to_abi()) };
            // Add additional code here to handle exceptions.
        }
    }

private:
    IAsyncAction SendMessageUsingStreamWebSocket(std::vector< byte >
message)
    {
        try
        {
            DataWriter dataWriter{ m_streamWebSocket.OutputStream() };
            dataWriter.WriteBytes(message);

            co_await dataWriter.StoreAsync();
            dataWriter.DetachStream();
            std::wstringstream wstringstream;
            wstringstream << L"Sending data using StreamWebSocket: " <<
message.size() << L" bytes" << std::endl;
            ::OutputDebugString(wstringstream.str().c_str());
        }
        catch (winrt::hresult_error const& ex)

```

```

    {
        Windows::Web::WebErrorStatus webErrorStatus{
Windows::Networking::Sockets::WebSocketError::GetStatus(ex.to_abi()) };
        // Add additional code here to handle exceptions.
    }
}

IAsyncAction ReceiveMessageUsingStreamWebSocket()
{
    try
    {
        DataReader dataReader{ m_streamWebSocket.InputStream() };
        dataReader.InputStreamOptions(InputStreamOptions::Partial);

        unsigned int bytesLoaded = co_await dataReader.LoadAsync(256);
        std::vector< byte > message(bytesLoaded);
        dataReader.ReadBytes(message);
        std::wstringstream wstringstream;
        wstringstream << L"Data received from StreamWebSocket: " <<
message.size() << " bytes" << std::endl;
        ::OutputDebugString(wstringstream.str().c_str());
        m_streamWebSocket.Close(1000, L"");
    }
    catch (winrt::hresult_error const& ex)
    {
        Windows::Web::WebErrorStatus webErrorStatus{
Windows::Networking::Sockets::WebSocketError::GetStatus(ex.to_abi()) };
        // Add additional code here to handle exceptions.
    }
}

void OnWebSocketClosed(Windows::Networking::Sockets::IWebSocket const&,
Windows::Networking::Sockets::WebSocketClosedEventArgs const& args)
{
    std::wstringstream wstringstream;
    wstringstream << L"WebSocket_Closed; Code: " << args.Code() << ",
Reason: \"" << args.Reason().c_str() << "\"\" << std::endl;
    ::OutputDebugString(wstringstream.str().c_str());
    // Add additional code here to handle the WebSocket being closed.
}

```

C++

```

#include <ppltasks.h>
#include <sstream>

...
using namespace Windows::Foundation;
using namespace Windows::Storage::Streams;
using namespace Windows::UI::Xaml::Navigation;
...
private:
    Windows::Networking::Sockets::StreamWebSocket^ streamWebSocket;

```

```

protected:
    virtual void OnNavigatedTo(NavigationEventArgs^ e) override
    {
        this->streamWebSocket = ref new
Windows::Networking::Sockets::StreamWebSocket();

        this->streamWebSocket->Closed += ref new
TypedEventHandler<Windows::Networking::Sockets::IWebSocket^,
Windows::Networking::Sockets::WebSocketClosedEventArgs^>(this,
&StreamWebSocketPage::WebSocket_Closed);

        try
        {
            auto connectTask = Concurrency::create_task(this-
>streamWebSocket->ConnectAsync(ref new Uri(L"wss://echo.websocket.org")));

            connectTask.then(
                [=]
                {
                    this->ReceiveMessageUsingStreamWebSocket();
                    this->SendMessageUsingStreamWebSocket(ref new
Platform::Array< byte >{ 0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07,
0x08, 0x09 });
                });
        }
        catch (Platform::Exception^ ex)
        {
            Windows::Web::WebErrorStatus webErrorStatus =
Windows::Networking::Sockets::WebSocketError::GetStatus(ex->HResult);
            // Add additional code here to handle exceptions.
        }
    }

private:
    void SendMessageUsingStreamWebSocket(const Platform::Array< byte >^
message)
    {
        try
        {
            auto dataWriter = ref new DataWriter(this->streamWebSocket-
>OutputStream);
            dataWriter->WriteBytes(message);

            Concurrency::create_task(dataWriter->StoreAsync()).then(
                [=](Concurrency::task< unsigned int >) // task< unsigned int
> instead of unsigned int in order to handle any exceptions thrown in
StoreAsync().
                {
                    dataWriter->DetachStream();
                    std::wstringstream wstringstream;
                    wstringstream << L"Sending data using StreamWebSocket: " <<
message->Length << L" bytes" << std::endl;
                    ::OutputDebugString(wstringstream.str().c_str());
                });
        }
    }

```

```

        catch (Platform::Exception^ ex)
        {
            Windows::Web::WebErrorStatus webErrorStatus =
Windows::Networking::Sockets::WebSocketError::GetStatus(ex->HResult);
            // Add additional code here to handle exceptions.
        }
    }

    void ReceiveMessageUsingStreamWebSocket()
    {
        try
        {
            DataReader^ dataReader = ref new DataReader(this-
>streamWebSocket->InputStream);
            dataReader->InputStreamOptions = InputStreamOptions::Partial;

            Concurrency::create_task(dataReader->LoadAsync(256)).then(
                [=](unsigned int bytesLoaded)
                {
                    auto message = ref new Platform::Array< byte >(bytesLoaded);
                    dataReader->ReadBytes(message);
                    std::wstringstream wstringstream;
                    wstringstream << L"Data received from StreamWebSocket: " <<
message->Length << " bytes" << std::endl;
                    ::OutputDebugString(wstringstream.str().c_str());
                    this->streamWebSocket->Close(1000, L"");
                });
        }
        catch (Platform::Exception^ ex)
        {
            Windows::Web::WebErrorStatus webErrorStatus =
Windows::Networking::Sockets::WebSocketError::GetStatus(ex->HResult);
            // Add additional code here to handle exceptions.
        }
    }

    void WebSocket_Closed(Windows::Networking::Sockets::IWebSocket^ sender,
Windows::Networking::Sockets::WebSocketClosedEventArgs^ args)
    {
        std::wstringstream wstringstream;
        wstringstream << L"WebSocket_Closed; Code: " << args->Code << ",
Reason: \"" << args->Reason->Data() << "\"" << std::endl;
        ::OutputDebugString(wstringstream.str().c_str());
        // Add additional code here to handle the WebSocket being closed.
    }
}

```

Handle the StreamWebSocket.Closed event

Before establishing a connection and sending data with a **StreamWebSocket**, you should subscribe to the **StreamWebSocket.Closed** event. **Closed** is raised when the client or the server closes the socket.

Send data on a StreamWebSocket

Once a connection is established, you can send data to the server. You do this by using the [StreamWebSocket.OutputStream](#) property, and a [DataWriter](#), to write the data.

Note If you want to write more data on the same socket, then be sure to call [DataWriter.DetachStream](#) to detach the output stream from the **DataWriter** before the **DataWriter** goes out of scope. This returns ownership of the stream to the **MessageWebSocket**.

Receive data on a StreamWebSocket

Use the [StreamWebSocket.InputStream](#) property, and a [DataReader](#), to read the data.

Advanced options for MessageWebSocket and StreamWebSocket

Before establishing a connection, you can set advanced options on a socket by setting properties on either [MessageWebSocketControl](#) or [StreamWebSocketControl](#). You access an instance of those classes from the socket object itself either via its [MessageWebSocket.Control](#) property or its [StreamWebSocket.Control](#) property, as appropriate.

Here's an example using **StreamWebSocket**. The same pattern applies to **MessageWebSocket**.

C#

```
var streamWebSocket = new Windows.Networking.Sockets.StreamWebSocket();

// By default, the Nagle algorithm is not used. This overrides that, and
// causes it to be used.
streamWebSocket.Control.NoDelay = false;

await streamWebSocket.ConnectAsync(new Uri("wss://echo.websocket.org"));
```

C++/WinRT

```
Windows::Networking::Sockets::StreamWebSocket streamWebSocket;

// By default, the Nagle algorithm is not used. This overrides that, and
// causes it to be used.
streamWebSocket.Control().NoDelay(false);
```

```
auto connectAsyncAction = streamWebSocket.ConnectAsync(Uri{
    L"wss://echo.websocket.org" });
```

C++

```
auto streamWebSocket = ref new
Windows::Networking::Sockets::StreamWebSocket();

// By default, the Nagle algorithm is not used. This overrides that, and
// causes it to be used.
streamWebSocket->Control->NoDelay = false;

auto connectTask = Concurrency::create_task(streamWebSocket->
    ConnectAsync(ref new Uri(L"wss://echo.websocket.org")));
```

Note Don't try to change a control property *after* you've called **ConnectAsync**. The only exception to that rule is [MessageWebSocketControl.MessageType](#).

WebSocket information classes

[MessageWebSocket](#) and [StreamWebSocket](#) each have a corresponding class that provides additional information about the object.

[MessageWebSocketInformation](#) provides information about a **MessageWebSocket**, and you retrieve an instance of it using the [MessageWebSocket.Information](#) property.

[StreamWebSocketInformation](#) provides information about a **StreamWebSocket**, and you retrieve an instance of it using the [StreamWebSocket.Information](#) property.

Note that the properties on these information classes are read-only, but you can use them to retrieve information at any time during the lifetime of a web socket object.

Handling exceptions

An error encountered on a [MessageWebSocket](#) or [StreamWebSocket](#) operation is returned as an **HRESULT** value. You can pass that **HRESULT** value to the [WebSocketError.GetStatus](#) method to convert it into a [WebErrorStatus](#) enumeration value.

Most **WebErrorStatus** enumeration values correspond to an error returned by the native HTTP client operation. Your app can switch on **WebErrorStatus** enumeration values to modify app behavior depending on the cause of the exception.

For parameter validation errors, you can use the **HRESULT** from the exception to learn more detailed information about the error. Possible **HRESULT** values are listed in `Winerror.h`, which can be found in your SDK installation (for example, in the folder `C:\Program Files (x86)\Windows Kits\10\Include\<VERSION>\shared`). For most parameter validation errors, the **HRESULT** returned is **E_INVALIDARG**.

Setting timeouts on WebSocket operations

MessageWebSocket and **StreamWebSocket** use an internal system service to send WebSocket client requests, and to receive responses from a server. The default timeout value used for a WebSocket connect operation is 60 seconds. If the HTTP server that supports WebSockets doesn't or can't respond to the WebSocket connection request (it's temporarily down, or blocked by a network outage), then the internal system service waits the default 60 seconds before it returns an error. That error causes an exception to be thrown on the WebSocket **ConnectAsync** method. For send and receive operations after a WebSocket connection has been established, the default timeout is 30 seconds.

If the name query for an HTTP server name in the URI returns multiple IP addresses for the name, then the internal system service tries up to 5 IP addresses for the site (each with a default timeout of 60 seconds) before it fails. Consequently, your app could wait several minutes trying to connect to multiple IP addresses before it handles an exception. This behavior might appear to the user like the app has stopped working.

To make your app more responsive and minimize these issues, you can set a shorter timeout on connection requests. You set a timeout in a similar way for both **MessageWebSocket** and **StreamWebSocket**.

C#

```
private Windows.Networking.Sockets.MessageWebSocket messageWebSocket;

protected override void OnNavigatedTo(NavigationEventArgs e)
{
    this.messageWebSocket = new
Windows.Networking.Sockets.MessageWebSocket();

    try
    {
        var cancellationTokenSource = new CancellationTokensource();
        var connectTask = this.messageWebSocket.ConnectAsync(new
Uri("wss://echo.websocket.org")).AsTask(cancellationTokenSource.Token);

        // Cancel connectTask after 5 seconds.

        cancellationTokenSource.CancelAfter(TimeSpan.FromMilliseconds(5000));
    }
}
```

```

connectTask.ContinueWith((antecedent) =>
{
    if (antecedent.Status == TaskStatus.RanToCompletion)
    {
        // connectTask ran to completion, so we know that the
        MessageWebSocket is connected.
        // Add additional code here to use the MessageWebSocket.
    }
    else
    {
        // connectTask timed out, or faulted.
    }
});
}
catch (Exception ex)
{
    Windows.Web.WebErrorStatus webErrorStatus =
Windows.Networking.Sockets.WebSocketError.GetStatus(ex.GetBaseException().HR
esult);
    // Add additional code here to handle exceptions.
}
}

```

C++/WinRT

```

#include <winrt/Windows.Foundation.h>
#include <winrt/Windows.Networking.Sockets.h>
#include <winrt/Windows.UI.Xaml.Navigation.h>
#include <sstream>

using namespace winrt;
using namespace Windows::Foundation;
using namespace Windows::UI::Xaml::Navigation;
...
private:
    Windows::Networking::Sockets::MessageWebSocket m_messageWebSocket;

    IAsyncAction TimeoutAsync()
    {
        // Return control to the caller, and resume again to complete the
        async action after the timeout period.
        // 5 seconds, in this example.
        co_await(std::chrono::seconds{ 5 });
    }

    public:
    IAsyncAction OnNavigatedTo(NavigationEventArgs /* e */)
    {
        try
        {
            // Return control to the caller, and then immediately resume on
            a thread pool thread.
            co_await winrt::resume_background();
        }
    }
}

```

```

        auto connectAsyncAction = m_messageWebSocket.ConnectAsync(Uri{
L"wss://echo.websocket.org" });

        TimeoutAsync().Completed([connectAsyncAction](IAsyncAction
const& sender, AsyncStatus const)
        {
            // TimeoutAsync completes after the timeout period. After
that period, it's safe
            // to cancel the ConnectAsync action even if it has already
completed.
            connectAsyncAction.Cancel();
        });

        try
        {
            // Block until the ConnectAsync action completes or is
canceled.
            connectAsyncAction.get();
        }
        catch (winrt::hresult_error const& ex)
        {
            std::wstringstream wstringstream;
            wstringstream << L"ConnectAsync threw an exception: " <<
ex.message().c_str() << std::endl;
            ::OutputDebugString(wstringstream.str().c_str());
        }

        if (connectAsyncAction.Status() == AsyncStatus::Completed)
        {
            // connectTask ran to completion, so we know that the
MessageWebSocket is connected.
            // Add additional code here to use the MessageWebSocket.
        }
        else
        {
            // connectTask did not run to completion.
        }
    }
    catch (winrt::hresult_error const& ex)
    {
        Windows::Web::WebErrorStatus webErrorStatus{
Windows::Networking::Sockets::WebSocketError::GetStatus(ex.to_abi()) };
        // Add additional code here to handle exceptions.
    }
}

```

C++

```

#include <agents.h>
#include <ppltasks.h>
#include <sstream>
...

```

```

using namespace Windows::Foundation;
using namespace Windows::Storage::Streams;
using namespace Windows::UI::Xaml::Navigation;
...
private:
    Windows::Networking::Sockets::MessageWebSocket^ messageWebSocket;

protected:
    virtual void OnNavigatedTo(NavigationEventArgs^ e) override
    {
        this->messageWebSocket = ref new
Windows::Networking::Sockets::MessageWebSocket();

        try
        {
            Concurrency::cancellation_token_source cancellationTokenSource;
            Concurrency::cancellation_token cancellationToken =
cancellationTokenSource.get_token();

            auto connectTask = Concurrency::create_task(this-
>messageWebSocket->ConnectAsync(ref new Uri(L"wss://echo.websocket.org")),
cancellationToken);

            // This continuation task returns true should connectTask run to
completion.
            Concurrency::task< bool > taskRanToCompletion =
connectTask.then([](void)
            {
                return true;
            });

            // This task returns false after the specified timeout. 5
seconds, in this example.
            Concurrency::task< bool > taskTimedout =
Concurrency::create_task([]() -> bool
            {
                Concurrency::task_completion_event< void >
taskCompletionEvent;

                // A call object that sets the task completion event.
                auto call = std::make_shared< Concurrency::call< int > >
([taskCompletionEvent](int)
                {
                    taskCompletionEvent.set();
                });

                // A non-repeating timer that calls the call object when the
timer fires.
                auto nonRepeatingTimer = std::make_shared<
Concurrency::timer< int > >(5000, 0, call.get(), false);
nonRepeatingTimer->start();

                // A task that completes after the completion event is set.
                Concurrency::task< void >
taskWaitForCompletionEvent(taskCompletionEvent);

```

```

        return taskWaitForCompletionEvent.then([]() {return false;
    }).get();
    });

    (taskRanToCompletion || taskTimedout).then([this,
cancellationTokenSource](bool connectTaskRanToCompletion)
    {
        if (connectTaskRanToCompletion)
        {
            // connectTask ran to completion, so we know that the
MessageWebSocket is connected.
            // Add additional code here to use the MessageWebSocket.
        }
        else
        {
            // taskTimedout ran to completion, so we should cancel
connectTask via the cancellation_token_source.
            cancellationTokenSource.cancel();
        }
    });
}
catch (Platform::Exception^ ex)
{
    Windows::Web::WebErrorStatus webErrorStatus =
Windows::Networking::Sockets::WebSocketError::GetStatus(ex->HResult);
    // Add additional code here to handle exceptions.
}
}

```

Important APIs

- [DataReader](#)
- [DataWriter](#)
- [DataWriter.DetachStream](#)
- [MessageWebSocket](#)
- [MessageWebSocket.Closed](#)
- [MessageWebSocket.ConnectAsync](#)
- [MessageWebSocket.Control](#)
- [MessageWebSocket.Information](#)
- [MessageWebSocket.MessageReceived](#)
- [MessageWebSocket.OutputStream](#)
- [MessageWebSocketControl](#)
- [MessageWebSocketControl.MessageType](#)
- [MessageWebSocketInformation](#)
- [MessageWebSocketMessageReceivedEventArgs](#)
- [SocketMessageType](#)

- [StreamWebSocket](#)
- [StreamWebSocket.Closed](#)
- [StreamSocket.ConnectAsync](#)
- [StreamWebSocket.Control](#)
- [StreamWebSocket.Information](#)
- [StreamWebSocket.InputStream](#)
- [StreamWebSocket.OutputStream](#)
- [StreamWebSocketControl](#)
- [StreamWebSocketInformation](#)
- [WebErrorStatus](#)
- [WebSocketError.GetStatus](#)
- [Windows.Networking.Sockets](#)

Related topics

- [WebSocket Protocol](#) 
- [Sockets](#)

Samples

- [WebSocket sample](#) 

HttpClient

Article • 10/20/2022

Important APIs

- [HttpClient](#)
- [Windows.Web.Http](#)
- [Windows.Web.Http.HttpResponseMessage](#)

Use [HttpClient](#) and the rest of the [Windows.Web.Http](#) namespace API to send and receive information using the HTTP 2.0 and HTTP 1.1 protocols.

Overview of HttpClient and the Windows.Web.Http namespace

The classes in the [Windows.Web.Http](#) namespace and the related [Windows.Web.Http.Headers](#) and [Windows.Web.Http.Filters](#) namespaces provide a programming interface for Universal Windows Platform (UWP) apps that act as an HTTP client to perform basic GET requests or implement more advanced HTTP functionality listed below.

- Methods for common verbs (**DELETE**, **GET**, **PUT**, and **POST**). Each of these requests are sent as an asynchronous operation.
- Support for common authentication settings and patterns.
- Access to Secure Sockets Layer (SSL) details on the transport.
- Ability to include customized filters in advanced apps.
- Ability to get, set, and delete cookies.
- HTTP Request progress info available on asynchronous methods.

The [Windows.Web.Http.HttpRequestMessage](#) class represents an HTTP request message sent by [Windows.Web.Http.HttpClient](#). The [Windows.Web.Http.HttpResponseMessage](#) class represents an HTTP response message received from an HTTP request. HTTP messages are defined in [RFC 2616](#) [↗](#) by the IETF.

The [Windows.Web.Http](#) namespace represents HTTP content as the HTTP entity body and headers including cookies. HTTP content can be associated with an HTTP request or

an HTTP response. The **Windows.Web.Http** namespace provides a number of different classes to represent HTTP content.

- **HttpBufferContent**. Content as a buffer
- **HttpFormUrlEncodedContent**. Content as name and value tuples encoded with the **application/x-www-form-urlencoded** MIME type
- **HttpMultipartContent**. Content in the form of the **multipart/*** MIME type.
- **HttpMultipartFormDataContent**. Content that is encoded as the **multipart/form-data** MIME type.
- **HttpStreamContent**. Content as a stream (the internal type is used by the HTTP GET method to receive data and the HTTP POST method to upload data)
- **HttpStringContent**. Content as a string.
- **IHttpContent** - A base interface for developers to create their own content objects

The code snippet in the "Send a simple GET request over HTTP" section uses the **HttpStringContent** class to represent the HTTP response from an HTTP GET request as a string.

The **Windows.Web.Http.Headers** namespace supports creation of HTTP headers and cookies, which are then associated as properties with **HttpRequestMessage** and **HttpResponseMessage** objects.

Send a simple GET request over HTTP

As mentioned earlier in this article, the **Windows.Web.Http** namespace allows UWP apps to send GET requests. The following code snippet demonstrates how to send a GET request to `http://www.contoso.com` using the **Windows.Web.Http.HttpClient** class and the **Windows.Web.Http.HttpResponseMessage** class to read the response from the GET request.

C#

```
//Create an HTTP client object
Windows.Web.Http.HttpClient httpClient = new Windows.Web.Http.HttpClient();

//Add a user-agent header to the GET request.
var headers = httpClient.DefaultRequestHeaders;

//The safe way to add a header value is to use the TryParseAdd method and
verify the return value is true,
//especially if the header value is coming from user input.
string header = "ie";
if (!headers.UserAgent.TryParseAdd(header))
{
    throw new Exception("Invalid header value: " + header);
}
```

```

}

header = "Mozilla/5.0 (compatible; MSIE 10.0; Windows NT 6.2; WOW64;
Trident/6.0)";
if (!headers.UserAgent.TryParseAdd(header))
{
    throw new Exception("Invalid header value: " + header);
}

Uri requestUri = new Uri("http://www.contoso.com");

//Send the GET request asynchronously and retrieve the response as a string.
Windows.Web.Http.HttpResponseMessage httpResponse = new
Windows.Web.Http.HttpResponseMessage();
string httpResponseBody = "";

try
{
    //Send the GET request
    httpResponse = await httpClient.GetAsync(requestUri);
    httpResponse.EnsureSuccessStatusCode();
    httpResponseBody = await httpResponse.Content.ReadAsStringAsync();
}
catch (Exception ex)
{
    httpResponseBody = "Error: " + ex.HResult.ToString("X") + " Message: " +
ex.Message;
}

```

C++/WinRT

```

// pch.h
#pragma once
#include <winrt/Windows.Foundation.h>
#include <winrt/Windows.Web.Http.Headers.h>

// main.cpp : Defines the entry point for the console application.
#include "pch.h"
#include <iostream>
using namespace winrt;
using namespace Windows::Foundation;

int main()
{
    init_apartment();

    // Create an HttpClient object.
    Windows::Web::Http::HttpClient httpClient;

    // Add a user-agent header to the GET request.
    auto headers{ httpClient.DefaultRequestHeaders() };

    // The safe way to add a header value is to use the TryParseAdd method,

```

```

and verify the return value is true.
// This is especially important if the header value is coming from user
input.
std::wstring header{ L"ie" };
if (!headers.UserAgent().TryParseAdd(header))
{
    throw L"Invalid header value: " + header;
}

header = L"Mozilla/5.0 (compatible; MSIE 10.0; Windows NT 6.2; WOW64;
Trident/6.0)";
if (!headers.UserAgent().TryParseAdd(header))
{
    throw L"Invalid header value: " + header;
}

Uri requestUri{ L"http://www.contoso.com" };

// Send the GET request asynchronously, and retrieve the response as a
string.
Windows::Web::Http::HttpResponseMessage httpResponseMessage;
std::wstring httpResponseBody;

try
{
    // Send the GET request.
    httpResponseMessage = httpClient.GetAsync(requestUri).get();
    httpResponseMessage.EnsureSuccessStatusCode();
    httpResponseBody =
httpResponseMessage.Content().ReadAsStringAsync().get();
}
catch (winrt::hresult_error const& ex)
{
    httpResponseBody = ex.message();
}
std::wcout << httpResponseBody;
}

```

POST binary data over HTTP

The [C++/WinRT](#) code example below illustrates using form data and a POST request to send a small amount of binary data as a file upload to a web server. The code uses the [HttpBufferContent](#) class to represent the binary data, and the [HttpMultipartFormDataContent](#) class to represent the multi-part form data.

ⓘ Note

Calling **get** (as seen in the code example below) isn't appropriate for a UI thread. For the correct technique to use in that case, see [Concurrency and asynchronous](#)

C++/WinRT

```
// pch.h
#pragma once
#include <winrt/Windows.Foundation.h>
#include <winrt/Windows.Security.Cryptography.h>
#include <winrt/Windows.Storage.Streams.h>
#include <winrt/Windows.Web.Http.Headers.h>

// main.cpp : Defines the entry point for the console application.
#include "pch.h"
#include <iostream>
#include <sstream>
using namespace winrt;
using namespace Windows::Foundation;
using namespace Windows::Storage::Streams;

int main()
{
    init_apartment();

    auto buffer{
        Windows::Security::Cryptography::CryptographicBuffer::ConvertStringToBinary(
            L"A sentence of text to encode into binary to serve as sample
            data.",
            Windows::Security::Cryptography::BinaryStringEncoding::Utf8
        )
    };
    Windows::Web::Http::HttpBufferContent binaryContent{ buffer };
    // You can use the 'image/jpeg' content type to represent any binary
    data;
    // it's not necessarily an image file.
    binaryContent.Headers().Append(L"Content-Type", L"image/jpeg");

    Windows::Web::Http::Headers::HttpContentDispositionHeaderValue
    disposition{ L"form-data" };
    binaryContent.Headers().ContentDisposition(disposition);
    // The 'name' directive contains the name of the form field representing
    the data.
    disposition.Name(L"fileForUpload");
    // Here, the 'filename' directive is used to indicate to the server a
    file name
    // to use to save the uploaded data.
    disposition.FileName(L"file.dat");

    Windows::Web::Http::HttpMultipartFormDataContent postContent;
    postContent.Add(binaryContent); // Add the binary data content as a part
    of the form data content.

    // Send the POST request asynchronously, and retrieve the response as a
```

```

string.
    Windows::Web::Http::HttpResponseMessage httpResponseMessage;
    std::wstring httpResponseBody;

    try
    {
        // Send the POST request.
        Uri requestUri{ L"https://www.contoso.com/post" };
        Windows::Web::Http::HttpClient httpClient;
        httpResponseMessage = httpClient.PostAsync(requestUri,
postContent).get();
        httpResponseMessage.EnsureSuccessStatusCode();
        httpResponseBody =
httpResponseMessage.Content().ReadAsStringAsync().get();
    }
    catch (winrt::hresult_error const& ex)
    {
        httpResponseBody = ex.message();
    }
    std::wcout << httpResponseBody;
}

```

To POST the contents of an actual binary file (rather than the explicit binary data used above), you'll find it easier to use an [HttpStreamContent](#) object. Construct one and, as the argument to its constructor, pass the value returned from a call to [StorageFile.OpenReadAsync](#). That method returns a stream for the data inside your binary file.

Also, if you're uploading a large file (larger than about 10MB), then we recommend that you use the Windows Runtime [Background Transfer](#) APIs.

POST JSON data over HTTP

The following example posts some JSON to an endpoint, then writes out the response body.

CS

```

using System;
using System.Diagnostics;
using System.Threading.Tasks;
using Windows.Storage.Streams;
using Windows.Web.Http;

private async Task TryPostJsonAsync()
{
    try
    {
        // Construct the HttpClient and Uri. This endpoint is for test

```

purposes only.

```
HttpClient httpClient = new HttpClient();
Uri uri = new Uri("https://www.contoso.com/post");

// Construct the JSON to post.
HttpStringContent content = new HttpStringContent(
    "{ \"firstName\": \"Eliot\" }",
    UnicodeEncoding.UTF8,
    "application/json");

// Post the JSON and wait for a response.
HttpResponseMessage httpResponseMessage = await
httpClient.PostAsync(
    uri,
    content);

// Make sure the post succeeded, and write out the response.
httpResponseMessage.EnsureSuccessStatusCode();
var httpResponseBody = await
httpResponseMessage.Content.ReadAsStringAsync();
Debug.WriteLine(httpResponseBody);
}
catch (Exception ex)
{
    // Write out any exceptions.
    Debug.WriteLine(ex);
}
}
```

C++/WinRT

```
// pch.h
#pragma once
#include <winrt/Windows.Foundation.h>
#include <winrt/Windows.Security.Cryptography.h>
#include <winrt/Windows.Storage.Streams.h>
#include <winrt/Windows.Web.Http.Headers.h>

// main.cpp : Defines the entry point for the console application.
#include "pch.h"
#include <iostream>
#include <sstream>
using namespace winrt;
using namespace Windows::Foundation;
using namespace Windows::Storage::Streams;

int main()
{
    init_apartment();

    Windows::Web::Http::HttpResponseMessage httpResponseMessage;
    std::wstring httpResponseBody;
```

```

try
{
    // Construct the HttpClient and Uri. This endpoint is for test
    purposes only.
    Windows::Web::Http::HttpClient httpClient;
    Uri requestUri{ L"https://www.contoso.com/post" };

    // Construct the JSON to post.
    Windows::Web::Http::HttpStringContent jsonContent(
        L"{ \"firstName\": \"Eliot\" }",
        UnicodeEncoding::Utf8,
        L"application/json");

    // Post the JSON, and wait for a response.
    HttpResponseMessage = httpClient.PostAsync(
        requestUri,
        jsonContent).get();

    // Make sure the post succeeded, and write out the response.
    HttpResponseMessage.EnsureSuccessStatusCode();
    HttpResponseMessage =
    HttpResponseMessage.Content().ReadAsStringAsync().get();
    std::wcout << HttpResponseMessage.c_str();
}
catch (winrt::hresult_error const& ex)
{
    std::wcout << ex.message().c_str();
}
}

```

Exceptions in Windows.Web.Http

An exception is thrown when an invalid string for a Uniform Resource Identifier (URI) is passed to the constructor for the [Windows.Foundation.Uri](#) object.

.NET: The [Windows.Foundation.Uri](#) type appears as [System.Uri](#) in C# and VB.

In C# and Visual Basic, this error can be avoided by using the [System.Uri](#) class in the .NET 4.5 and one of the [System.Uri.TryCreate](#) methods to test the string received from a user before the URI is constructed.

In C++, there is no method to try and parse a string to a URI. If an app gets input from the user for the [Windows.Foundation.Uri](#), the constructor should be in a try/catch block. If an exception is thrown, the app can notify the user and request a new hostname.

The [Windows.Web.Http](#) lacks a convenience function. So an app using [HttpClient](#) and other classes in this namespace needs to use the **HRESULT** value.

In apps using C++/WinRT, the `winrt::hresult_error` struct represents an exception raised during app execution. The `winrt::hresult_error::code` function returns the **HRESULT** assigned to the specific exception. The `winrt::hresult_error::message` function returns the system-provided string that is associated with the **HRESULT** value. For more info, see [Error handling with C++/WinRT](#)

Possible **HRESULT** values are listed in the *Winerror.h* header file. Your app can filter on specific **HRESULT** values to modify app behavior depending on the cause of the exception.

In apps using the .NET Framework 4.5 in C#, VB.NET, the [System.Exception](#) represents an error during app execution when an exception occurs. The [System.Exception.HResult](#) property returns the **HRESULT** assigned to the specific exception. The [System.Exception.Message](#) property returns the message that describes the exception.

C++/CX has been superseded by C++/WinRT. But in apps using C++/CX, the [Platform::Exception](#) represents an error during app execution when an exception occurs. The [Platform::Exception::HResult](#) property returns the **HRESULT** assigned to the specific exception. The [Platform::Exception::Message](#) property returns the system-provided string that is associated with the **HRESULT** value.

For most parameter validation errors, the **HRESULT** returned is **E_INVALIDARG**. For some illegal method calls, the **HRESULT** returned is **E_ILLEGAL_METHOD_CALL**.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

RSS/Atom feeds

Article • 10/26/2022

Important APIs

- [Windows.Data.Xml.Dom](#)
- [Windows.Web.AtomPub](#)
- [Windows.Web.Syndication](#)

Retrieve or create the most current and popular Web content using syndicated feeds generated according to the RSS and Atom standards using features in the [Windows.Web.Syndication](#) namespace.

What is a feed?

A web feed is a document that contains any number of individual entries made up of text, links, and images. Updates made to a feed are in the form of new entries used to promote the latest content across the Web. Content consumers can use a feed reader app to aggregate and monitor feeds from any number of individual content authors, gaining access to the latest content quickly and conveniently.

Which feed format standards are supported?

The Universal Windows Platform (UWP) supports feed retrieval for RSS format standards from 0.91 to RSS 2.0, and Atom standards from 0.3 to 1.0. Classes in the [Windows.Web.Syndication](#) namespace can define feeds and feed items capable of representing both RSS and Atom elements.

Additionally, Atom 1.0 and RSS 2.0 formats both allow their feed documents to contain elements or attributes not defined in the official specifications. Over time, these custom elements and attributes have become a way to define domain-specific information consumed by other web service data formats like GData and OData. To support this added feature, the [SyndicationNode](#) class represents generic XML elements. Using [SyndicationNode](#) with classes in the [Windows.Data.Xml.Dom](#) namespace, allows apps to access attributes, extensions, and any content that they may contain.

Note that, for publication of syndicated content, the UWP implementation of the Atom Publication Protocol ([Windows.Web.AtomPub](#)) only supports feed content operations according to the Atom and Atom Publication standards.

Using syndicated content with network isolation

The network isolation feature in the UWP enables a developer to control and limit network access by a UWP app. Not all apps may require access to the network. However for those apps that do, UWP provides different levels of access to the network that can be enabled by selecting appropriate capabilities.

Network isolation allows a developer to define for each app the scope of required network access. An app without the appropriate scope defined is prevented from accessing the specified type of network, and specific type of network request (outbound client-initiated requests or both inbound unsolicited requests and outbound client-initiated requests). The ability to set and enforce network isolation ensures that if an app does get compromised, it can only access networks where the app has explicitly been granted access. This significantly reduces the scope of the impact on other applications and on Windows.

Network isolation affects any class elements in the [Windows.Web.Syndication](#) and [Windows.Web.AtomPub](#) namespaces that try to access the network. Windows actively enforces network isolation. A call to a class element in the **Windows.Web.Syndication** or **Windows.Web.AtomPub** namespace that results in network access may fail because of network isolation if the appropriate network capability has not been enabled.

The network capabilities for an app are configured in the app manifest when the app is built. Network capabilities are usually added using Microsoft Visual Studio 2015 when developing the app. Network capabilities may also be set manually in the app manifest file using a text editor.

For more detailed information on network isolation and networking capabilities, see the "Capabilities" section in the [Networking basics](#) topic.

How to access a web feed

This section shows how to retrieve and display a web feed using classes in the [Windows.Web.Syndication](#) namespace in your UWP app written in C# or JavaScript.

Prerequisites

To ensure your UWP app is network ready, you must set any network capabilities that are needed in the project **Package.appxmanifest** file. If your app needs to connect as a client to remote services on the Internet, then the **internetClient** capability is needed. For more information, see the "Capabilities" section in the [Networking basics](#) topic.

Retrieving syndicated content from a web feed

Now we will review some code that demonstrates how to retrieve a feed, and then display each individual item that the feed contains. Before we can configure and send the request, we'll define a few variables we'll be using during the operation, and initialize an instance of [SyndicationClient](#), which defines the methods and properties we'll use to retrieve and display the feed.

The [Uri](#) constructor throws an exception if the *uriString* passed to the constructor is not a valid URI. So we validate the *uriString* using a try/catch block.

C#

```
Windows.Web.Syndication.SyndicationClient client = new
Windows.Web.Syndication.SyndicationClient();
Windows.Web.Syndication.SyndicationFeed feed;
// The URI is validated by catching exceptions thrown by the Uri
// constructor.
Uri uri = null;
// Use your own uriString for the feed you are connecting to.
string uriString = "";
try
{
    uri = new Uri(uriString);
}
catch (Exception ex)
{
    // Handle the invalid URI here.
}
```

JavaScript

```
var currentFeed = null;
var currentItemIndex = 0;
var client = new Windows.Web.Syndication.SyndicationClient();
// The URI is validated by catching exceptions thrown by the Uri
// constructor.
var uri = null;
try {
    uri = new Windows.Foundation.Uri(uriString);
} catch (error) {
    WinJS.log && WinJS.log("Error: Invalid URI");
    return;
}
```

Next we configure the request by setting any Server credentials (the [ServerCredential](#) property), proxy credentials (the [ProxyCredential](#) property), and HTTP headers (the [SetRequestHeader](#) method) needed. With the basic request parameters configured, a